

Nama: Fajar Satria Nusaputra
NIM: 2410651058
Kelas: Kamis-C

TUGAS 1: ANALISIS DEADLOCK

Mengapa program mengalami deadlock?

Program tersebut mengalami deadlock karena terdapat dua thread yang saling menunggu resource yang sedang dipegang oleh thread lainnya jadi nggak ada thread yang bisa lanjut contohnya di praktikum.

T1 memegang lock 1 terus tunggu lock 2 yang dipegang sama T2

T2 memegang lock 2 terus tunggu lock 1 yang dipegang sama T2 (gitu saja teruss, makanya jadi deadlock)

Kondisi deadlock apa saja yang terpenuhi?

Kondisi deadlock terpenuhi karena seluruh syaratnya udah ada di program itu

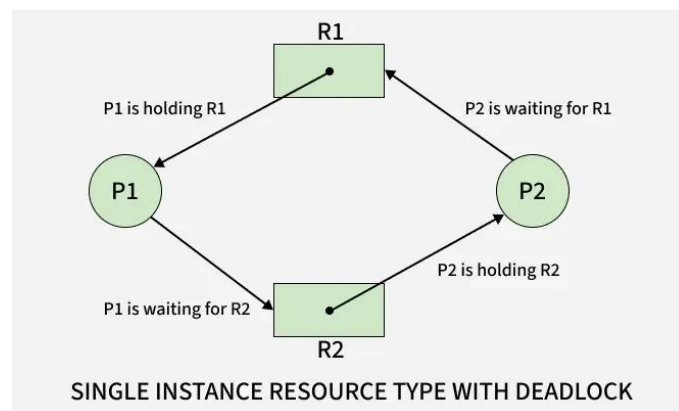
Mutual Exclusion: lock 1 sama lock 2 Cuma bisa dipegang satu thread dalam satu waktu

Hold and Wait: T1 pegang lock 1 sambil nunggu lock 2 terus T2 memegang lock 2 sambil nunggu lock 1

No Preemption: lock gabisa diambil paksa sama system jadi thread sendiri yg harus ngelepasin locknya

Circular Wait: terbentuk lingkaran saling tunggu, T1 nunggu T2 terus T2 nunggu T1

Gambar diagram resource allocation graphnya!



TUGAS 2: IMPLEMENTASI PENCEGAHAN

Jadi pada tugas kedua ini, kita akan membuat pencegahan deadlocknya, namun dengan dua teknik yang berbeda lalu membandingkan keduanya, yaitu Lock Ordering dan Try Lock. Cukup modifikasi program yang deadlock dengan kode berikut ini, masing-masing dengan urutan lock ordering dan try lock, tapi saya disini membuat program baru lagi di direktori tugas, lalu jalankan.

Lock Ordering:

```

#include <stdio.h>
#include <pthread.h>
#include <unistd.h>

pthread_mutex_t lock1 = PTHREAD_MUTEX_INITIALIZER;
pthread_mutex_t lock2 = PTHREAD_MUTEX_INITIALIZER;

void acquire_locks_in_order() {
    pthread_mutex_lock(&lock1);
    pthread_mutex_lock(&lock2);
}

void release_locks_in_order() {
    pthread_mutex_unlock(&lock2);
    pthread_mutex_unlock(&lock1);
}

void* thread1_func(void* arg) {
    printf("Thread 1: mulai\n");

    acquire_locks_in_order();
    printf("Thread 1: sedang menggunakan resource\n");
    sleep(1);
    release_locks_in_order();

    printf("Thread 1: selesai\n");
    return NULL;
}

void* thread2_func(void* arg) {
    printf("Thread 2: mulai\n");

    acquire_locks_in_order();
    printf("Thread 2: sedang menggunakan resource\n");
    sleep(1);
    release_locks_in_order();

    printf("Thread 2: selesai\n");
    return NULL;
}

int main() {
    pthread_t t1, t2;

    printf("=== PENCEGAHAN DEADLOCK: LOCK ORDERING ===\n\n");

    pthread_create(&t1, NULL, thread1_func, NULL);
    pthread_create(&t2, NULL, thread2_func, NULL);

    pthread_join(t1, NULL);
    pthread_join(t2, NULL);

    printf("\n=== SELESAI TANPA DEADLOCK ===\n");
    return 0;
}

```

Penjelasan

acquire_locks_in_order()

berfungsi mengatur urutan agar kedua thread selalu mengambil lock1 terlebih dahulu dan kemudian lock2 sehingga tidak terjadi proses saling tunggu. Kedua thread memanggil ini sebelum bekerja.

release_locks_in_order()

Berfungsi untuk melepas lock 2 terlebih dahulu, baru kemudian lock 1 untuk memastikan kedua resource Kembali dengan benar.

Try Lock:

```
#include <stdio.h>
#include <pthread.h>
#include <unistd.h>

pthread_mutex_t lock1 = PTHREAD_MUTEX_INITIALIZER;
pthread_mutex_t lock2 = PTHREAD_MUTEX_INITIALIZER;

void* thread1_func(void* arg) {
    int retry = 0;
    while (1) {
        pthread_mutex_lock(&lock1);

        if (pthread_mutex_trylock(&lock2) == 0) {
            printf("Thread 1: dapat kedua lock\n");

            sleep(1);

            pthread_mutex_unlock(&lock2);
            pthread_mutex_unlock(&lock1);
            break;
        } else {
            printf("Thread 1: gagal lock2, retry...\n");
            pthread_mutex_unlock(&lock1);
            retry++;
            usleep(100000); // 0.1 detik
        }
    }
    printf("Thread 1 selesai (retry: %d)\n", retry);
    return NULL;
}

void* thread2_func(void* arg) {
    int retry = 0;
    while (1) {
        pthread_mutex_lock(&lock2);

        if (pthread_mutex_trylock(&lock1) == 0) {
            printf("Thread 2: dapat kedua lock\n");
```

```

        sleep(1);

        pthread_mutex_unlock(&lock1);
        pthread_mutex_unlock(&lock2);
        break;
    } else {
        printf("Thread 2: gagal lock1, retry...\n");
        pthread_mutex_unlock(&lock2);
        retry++;
        usleep(150000); // 0.15 detik
    }
}
printf("Thread 2 selesai (retry: %d)\n", retry);
return NULL;
}

int main() {
    pthread_t t1, t2;

    printf("=== PENCEGAHAN DEADLOCK: TRY-LOCK ===\n\n");

    pthread_create(&t1, NULL, thread1_func, NULL);
    pthread_create(&t2, NULL, thread2_func, NULL);

    pthread_join(t1, NULL);
    pthread_join(t2, NULL);

    printf("\n=== SELESAI TANPA DEADLOCK ===\n");
    return 0;
}

```

Penjelasan:

pthread_mutex_trylock()

Fungsi try-lock tidak memungkinkan thread menunggu apabila lock dipegang oleh thread lain. Jika lock tidak tersedia, fungsi ini langsung gagal dan mengembalikan nilai non-zero. Oleh karena itu, thread tidak pernah berada pada kondisi "menunggu selamanya", yang merupakan salah satu syarat terjadinya deadlock. Ketika try-lock gagal, thread langsung melepaskan lock pertama yang ia pegang dengan menggunakan `pthread_mutex_unlock(&lock1)` atau `pthread_mutex_unlock(&lock2)`, lalu melakukan percobaan ulang setelah menunggu sebentar. Dalam proses ini, kondisi hold-and-wait dihilangkan karena tidak ada thread yang memegang lock satu sambil men Thread hanya memegang kunci ketika kedua kunci tersedia. Meskipun loop retry yang menggunakan `usleep()` tidak mencegah deadlock secara langsung, itu memberi waktu kepada thread lain untuk menyelesaikan tugasnya dan melepaskan lock.

Hasil:

```

PROGRAN DEEBOH
koch@Kchng:~$ nano lock_ordering.c
koch@Kchng:~$ gcc lock_ordering.c -o lock_ordering
koch@Kchng:~$ ./lock_ordering
=== PENCEGAHAN DEADLOCK: LOCK ORDERING ===

Thread 1: mulai
Thread 1: sedang menggunakan resource
Thread 2: mulai
Thread 1: selesai
Thread 2: sedang menggunakan resource
Thread 2: selesai

=== SELESAI TANPA DEADLOCK ===
koch@Kchng:~$ nano try_lock.c
koch@Kchng:~$ gcc try_lock.c -o try_lock
koch@Kchng:~$ ./try_lock
=== PENCEGAHAN DEADLOCK: TRY-LOCK ===

Thread 1: dapat kedua lock
Thread 2: dapat kedua lock
Thread 1 selesai (retry: 0)
Thread 2 selesai (retry: 0)

=== SELESAI TANPA DEADLOCK ===
koch@Kchng:~$ |

```

Bisa kita lihat pada hasil diatas, keduanya sama-sama berhasil mencegah deadlock, namun dengan cara yang berbeda. Pada Lock Ordering, program ini bekerja dengan cara memastikan semua thread mengambil resource dalam urutan yang sama sehingga memicu proses saling menunggu, sedangkan pada Try Lock, program bekerja dengan cara jika semisal thread mencoba mengambil lock kedua namun lock kedua sedang dipakai, thread akan melepas lock yang telah ia pegang lalu mencoba lagi nanti sehingga tidak ada thread yang memegang satu lock sambil menunggu lock lain yang mencegah kondisi hold and wait terjadi.

TUGAS 3: BANKER'S ALGORITHM

Disini, kita diperintahkan membuat program dengan ketentuan memiliki 4 proses, 3 jenis resource, available diinisialisasi sebagai {5, 4, 3}, serta mengimplementasi request resource dan cek keamanan sistem. Pertama-tama, disini saya membuat program bernama dengan isi kodenya sebagai berikut, lalu jalankan dan hasilnya seperti pada gambar dibawah kode.

Kode:

```

#include <stdio.h>
#include <stdbool.h>

#define P 4
#define R 3

int available[R] = {5, 4, 3};

data lain)
int maximum[P][R] = {
    {4, 3, 3},
    {2, 2, 2},
    {3, 3, 2},
    {4, 2, 2}
};

int allocation[P][R] = {
    {0, 1, 1},
    {2, 0, 0},
    {1, 1, 1},
    {0, 0, 2}
};

int need[P][R];

void calculate_need() {
    for (int i = 0; i < P; i++) {
        for (int j = 0; j < R; j++) {
            need[i][j] = maximum[i][j] - allocation[i][j];
        }
    }
}

void print_status() {
    printf("\n=== STATUS SISTEM ===\n");

    printf("Available: ");
    for (int i = 0; i < R; i++) printf("%d ", available[i]);
    printf("\n\nAllocation:\n");
    for (int i = 0; i < P; i++) {
        printf("P%d: ", i);
        for (int j = 0; j < R; j++) printf("%d ", allocation[i][j]);
        printf("\n");
    }

    printf("\nNeed:\n");
    for (int i = 0; i < P; i++) {
        printf("P%d: ", i);
        for (int j = 0; j < R; j++) printf("%d ", need[i][j]);
        printf("\n");
    }
}

bool is_safe() {

```

```

bool is_safe() {
    int work[R];
    bool finish[P] = {false};
    int safe_sequence[P];
    int count = 0;

    for (int i = 0; i < R; i++) {
        work[i] = available[i];
    }

    printf("\n=== MENGECEK SAFE STATE ===\n");

    while (count < P) {
        bool found = false;

        for (int i = 0; i < P; i++) {
            if (!finish[i]) {
                bool can_alloc = true;

                for (int j = 0; j < R; j++) {
                    if (need[i][j] > work[j]) {
                        can_alloc = false;
                        break;
                    }
                }

                if (can_alloc) {
                    for (int j = 0; j < R; j++) {
                        work[j] += allocation[i][j];
                    }
                    safe_sequence[count++] = i;
                    finish[i] = true;
                    found = true;
                }
            }
        }

        if (!found) {
            printf("Sistem TIDAK aman!\n");
            return false;
        }
    }

    printf("Sistem AMAN. Safe sequence: ");
    for (int i = 0; i < P; i++) {
        printf("%d ", safe_sequence[i]);
    }
    printf("\n");

    return true;
}

bool request_resources(int process, int request[]) {
    printf("\n=== REQUEST P%d ===\n", process);
    // ...
}

```



```

printf("Meminta: ");
for (int i = 0; i < R; i++) printf("%d ", request[i]);
printf("\n");

for (int i = 0; i < R; i++) {
    if (request[i] > need[process][i]) {
        printf("Ditolak: request > need\n");
        return false;
    }
}

for (int i = 0; i < R; i++) {
    if (request[i] > available[i]) {
        printf("Ditolak: resource tidak cukup\n");
        return false;
    }
}

for (int i = 0; i < R; i++) {
    available[i] -= request[i];
    allocation[process][i] += request[i];
    need[process][i] -= request[i];
}

if (is_safe()) {
    printf("Request DITERIMA\n");
    return true;
} else {
    printf("Request DITOLAK (unsafe), rollback...\n");
    for (int i = 0; i < R; i++) {
        available[i] += request[i];
        allocation[process][i] -= request[i];
        need[process][i] += request[i];
    }
    return false;
}
}

int main() {
    calculate_need();
    print_status();

    int req1[R] = {1, 0, 1};
    request_resources(0, req1);
    print_status();

    int req2[R] = {3, 3, 0};
    request_resources(2, req2);

    return 0;
}

```

Hasil:

```
koch@Kchng:~$ nano tugas_bankers.c
koch@Kchng:~$ gcc tugas_bankers.c -o tugas_banker
koch@Kchng:~$ ./tugas_bankers

=== STATUS SISTEM ===
Available: 5 4 3

Allocation:
P0: 0 1 1
P1: 2 0 0
P2: 1 1 1
P3: 0 0 2

Need:
P0: 4 2 2
P1: 0 2 2
P2: 2 2 1
P3: 4 2 0

=== REQUEST P0 ===
Meminta: 1 0 1

=== MENGECEK SAFE STATE ===
Sistem AMAN. Safe sequence: P0 P1 P2 P3
Request DITERIMA

=== STATUS SISTEM ===
Available: 4 4 2

Allocation:
P0: 1 1 2
P1: 2 0 0
P2: 1 1 1
P3: 0 0 2

Need:
P0: 3 2 1
P1: 0 2 2
P2: 2 2 1
P3: 4 2 0

=== REQUEST P2 ===
Meminta: 3 3 0
Ditolak: request > need
koch@Kchng:~$ |
```

Berdasarkan hasilnya pada gambar diatas, bis akita lihat bahwa sistem awal berada dalam kondisi aman dibuktikan dengan adanya *safe sequence* yang berhasil ditemukan, yaitu urutan eksekusi proses P1 → P3 → P4 → P0 → P2. Urutan ini menunjukkan bahwa seluruh proses dapat dijalankan secara berurutan tanpa menyebabkan deadlock, karena setiap proses memperoleh jumlah resource yang diperlukan setelah proses sebelumnya selesai dan mengembalikan resource miliknya. Pada akhir, permintaan ditolak karena resource yang diminta melebihi resource yang tersedia.

TUGAS 4: KASUS NYATA

Disini kita diperintahkan membuat simulasi suatu kasus dimana dua orang ingin transfer uang antar rekening secara bersamaan dengan ketentuan orang A transfer dari rekening 1 ke rekening 2 sedangkan orang B transfer dari rekening 2 ke rekening 1. Disini saya menggunakan teknik lock ordering. Pertama-tama, buat program deadlocknya terlebih dahulu kemudian buat program

```
#include <stdio.h>
#include <pthread.h>
#include <unistd.h>

typedef struct {
    int id;
    int balance;
    pthread_mutex_t lock;
} Account;

Account acc1 = { .id = 1, .balance = 100, .lock = PTHREAD_MUTEX_INITIALIZER };
Account acc2 = { .id = 2, .balance = 100, .lock = PTHREAD_MUTEX_INITIALIZER };

void transfer(Account *from, Account *to, int amount) {
    // ambil lock from dulu, lalu lock to (urutan berbeda di thread lain -> deadlock)
    printf("Thread %ld: mencoba lock account %d\n", pthread_self(), from->id);
    pthread_mutex_lock(&from->lock);
    printf("Thread %ld: terkunci account %d\n", pthread_self(), from->id);

    sleep(1); // beri waktu agar thread lain mengambil lock lain -> memicu deadlock

    printf("Thread %ld: mencoba lock account %d\n", pthread_self(), to->id);
    pthread_mutex_lock(&to->lock);
    printf("Thread %ld: terkunci account %d\n", pthread_self(), to->id);

    // lakukan transfer
    if (from->balance >= amount) {
        from->balance -= amount;
        to->balance += amount;
        printf("Thread %ld: transfer %d dari %d ke %d berhasil\n",
            pthread_self(), amount, from->id, to->id);
    } else {
        printf("Thread %ld: saldo tidak cukup\n", pthread_self());
    }

    pthread_mutex_unlock(&to->lock);
    pthread_mutex_unlock(&from->lock);
}

void* personA(void* arg) {
    // A: transfer dari acc1 ke acc2
    transfer(&acc1, &acc2, 10);
    return NULL;
}

void* personB(void* arg) {
```

```

        // B: transfer dari acc2 ke acc1
        transfer(&acc2, &acc1, 20);
        return NULL;
    }

int main() {
    pthread_t t1, t2;

    printf("Saldo awal: acc1=%d, acc2=%d\n", acc1.balance, acc2.balance);

    pthread_create(&t1, NULL, personA, NULL);
    pthread_create(&t2, NULL, personB, NULL);

    pthread_join(t1, NULL);
    pthread_join(t2, NULL);

    printf("Saldo akhir: acc1=%d, acc2=%d\n", acc1.balance, acc2.balance);
    return 0;
}

```

preventionnya dengan isi kode masing-masing secara berurutan sebagai berikut, kemudian jalankan.Prevention:

```

#include <stdio.h>
#include <pthread.h>
#include <unistd.h>

typedef struct {
    int id;
    int balance;
    pthread_mutex_t lock;
} Account;

Account acc1 = { .id = 1, .balance = 100, .lock = PTHREAD_MUTEX_INITIALIZER };
Account acc2 = { .id = 2, .balance = 100, .lock = PTHREAD_MUTEX_INITIALIZER };

void msleep(long ms) {
    struct timespec ts;
    ts.tv_sec = ms / 1000;
    ts.tv_nsec = (ms % 1000) * 1000000;
    nanosleep(&ts, NULL);
}

void transfer_trylock(Account *from, Account *to, int amount) {
    int retries = 0;

    while (1) {
        // ambil lock pertama (blocking)
        pthread_mutex_lock(&from->lock);

        // coba ambil lock kedua tanpa menunggu lama
        if (pthread_mutex_trylock(&to->lock) == 0) {
            // sukses: lakukan transfer
            if (from->balance >= amount) {
                from->balance -= amount;
                to->balance += amount;
                printf("Transfer %d dari %d ke %d berhasil (retries=%d)\n",
                    amount, from->id, to->id, retries);
            }
        }
    }
}

```

```

        amount, from->id, to->id, retries);
    } else {
        printf("Saldo tidak cukup untuk transfer %d dari %d ke %d\n",
            amount, from->id, to->id);
    }

    pthread_mutex_unlock(&to->lock);
    pthread_mutex_unlock(&from->lock);
    break;
} else {

    pthread_mutex_unlock(&from->lock);
    retries++;
    msleep(100);
}
}
}

void* personA(void* arg) {
    transfer_trylock(&acc1, &acc2, 10);
    return NULL;
}

void* personB(void* arg) {
    transfer_trylock(&acc2, &acc1, 20);
    return NULL;
}

int main() {
    pthread_t t1, t2;
    printf("Saldo awal: acc1=%d, acc2=%d\n", acc1.balance, acc2.balance);

    pthread_create(&t1, NULL, personA, NULL);
    pthread_create(&t2, NULL, personB, NULL);

    pthread_join(t1, NULL);
    pthread_join(t2, NULL);

    printf("Saldo akhir: acc1=%d, acc2=%d\n", acc1.balance, acc2.balance);
    return 0;
}

```

Hasil:

```
koch@Kchng:~$ nano deadlock_transfer.c
koch@Kchng:~$ gcc deadlock_transfer.c -o deadlock_transfer
koch@Kchng:~$ ./deadlock_transfer
Saldo awal: acc1=100, acc2=100
Thread 126686431536704: mencoba lock account 2
Thread 126686431536704: terkunci account 2
Thread 126686439929408: mencoba lock account 1
Thread 126686439929408: terkunci account 1
Thread 126686431536704: mencoba lock account 1
Thread 126686439929408: mencoba lock account 2
^C
koch@Kchng:~$ nano prevention_transfer.c
koch@Kchng:~$ gcc prevention_transfer.c -o prevention_transfer
koch@Kchng:~$ ./prevention-transfer
-bash: ./prevention-transfer: No such file or directory
koch@Kchng:~$ ./prevention_transfer
Saldo awal: acc1=100, acc2=100
Transfer 10 dari 1 ke 2 berhasil (retries=0)
Transfer 20 dari 2 ke 1 berhasil (retries=0)
Saldo akhir: acc1=110, acc2=90
koch@Kchng:~$ |
```

Dari gambar diatas, bisa kita lihat bahwa deadlock diawal bisa teratasi diakhirnya menggunakan Teknik lock ordering, yaitu mengunci (lock) rekening berdasarkan urutan yang konsisten yang membuat kedua thread mengambil lock dalam urutan yang sama sehingga proses saling menunggu tidak terjadi dan deadlock tercegah, yaitu transfer 10 dari rekening akun orang A (acc1) ke rekening akun orang B (acc2) dan transfer 20 dari rekening akun orang B (acc2) ke rekening akun orang A (acc1) berhasil.